

Lab 07: Classless Inter-domain Routing (CIDR) and Static Routing in Omnet++

Duration: 1 hour 30 minutes | In-Lab Assessment

Tools: OMNeT++ and INET Framework

Objectives:

- Learn how to design subnets and assign IPv4 addresses (CIDR / netmask).
- Use `Ipv4NetworkConfigurator` to auto-assign addresses and add static routes.
- Create and run a simple multi-subnet topology (hosts, switch, routers) and verify routing using `ping`, `traceroute` (or equivalent apps) and TCP traffic (`TcpSessionApp`).
- Observe routing behaviour (default gateway, forwarding across multiple hops) and basic troubleshooting steps.

Short theory:

All the necessary theory for routing, subnets, routing protocols are provided in the previous labs. Please go through them first.

Important INET/OMNeT++ concepts for routing:

- **Router** is a prebuilt compound module that contains IPv4 forwarding logic and routing table(s).
- **Ipv4NetworkConfigurator** can automatically assign addresses and optionally add static routes for simple topologies.
- Interfaces on hosts/routers are named differently (e.g., `eth`, `ppp`); use the OMNeT++ inspector to find exact parameter names for your INET version.
- Use `omnetpp.ini` to set app destinations, addresses (if manual), and routing-related parameters like `configurator.assignAddresses` and `configurator.addStaticRoutes`.

Before the lab

- OMNeT++ and INET installed and built.
- Know how to open NED files and `omnetpp.ini` in the IDE.
- Be ready to inspect module parameters in the runtime inspector (right-click module → Inspect).
- If something fails, consult INET version's API (module/parameter names can change across INET releases).

Part A: Run the provided ARP example

1. In the Project Explorer open the example NED file inside the `inet examples` and `ARP` folder, which contains the `ARPTTest` network.

2. Open its `omnetpp.ini` (or the supplied INI fragment below).
3. Right-click on `omnetpp.ini` and choose `Run As → OMNeT++ Simulation`.
4. In Qtenv, start the simulation. Watch the packet flows. Use the console to inspect TCP session logs (client connecting to server). Use the inspector to view assigned IP addresses
5. Try ping across subnets Observe whether the traffic traverses routers correctly.

Part B: ARPTest.ned

```

1 //
2 // Copyright (C) 2006 OpenSim Ltd.
3 //
4 // SPDX-License-Identifier: LGPL-3.0-or-later
5 //
6
7 package inet.examples.ethernet.arptest;
8
9 import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
10 import inet.node.ethernet.EthernetSwitch;
11 import inet.node.inet.Router;
12 import inet.node.inet.StandardHost;
13 import ned.DatarateChannel;
14
15
16 network ARPTest
17 {
18     types:
19         channel fiberline extends DatarateChannel
20         {
21             delay = 1us;
22             datarate = 512Mbps;
23         }
24         channel ethline extends DatarateChannel
25         {
26             delay = 50ns;
27             datarate = 100Mbps;
28         }
29     submodules:
30         client: StandardHost {
31             @display("p=250,100;i=device/laptop");
32         }
33         host1: StandardHost {
34             @display("p=250,200;i=device/pc");
35         }
36         host2: StandardHost {
37             @display("p=250,300;i=device/pc");
38         }
39         switch: EthernetSwitch {
40             @display("p=400,200");
41         }
42         net: Router {
43             @display("p=700,200");

```

```

44     }
45     router: Router {
46         @display("p=550,200");
47     }
48     server: StandardHost {
49         @display("p=850,200;i=device/server");
50     }
51     configurator: Ipv4NetworkConfigurator {
52         @display("p=100,100;is=s");
53     }
54     connections:
55         client.ethg++ <--> ethline <--> switch.ethg++;
56         switch.ethg++ <--> ethline <--> host1.ethg++;
57         switch.ethg++ <--> ethline <--> host2.ethg++;
58         router.ethg++ <--> ethline <--> switch.ethg++;
59         router.pppg++ <--> fiberline <--> net.pppg++;
60         server.pppg++ <--> fiberline <--> net.pppg++;
61 }

```

Explanation of important parts in ARPTest.ned

- Router modules (`router`, `net`) represent IP routers. They have PPP ports (`pppg[]`) and Ethernet ports (`ethg[]`) depending on configuration.
- `StandardHost` is a compound host module providing NIC, IPv4 stack and app containers (`client/server`).
- `Ipv4NetworkConfigurator` can auto-assign addresses and optionally add static routes. It simplifies addressing for multi-subnet topologies.
- Connections: note `ethg++` and `pppg++` use the next available gate index; this is convenient when building topologies with many links.
- PPP links (`pppg`) are often used to model point-to-point routed links between routers (useful for /30 or /31 subnets).

Part C: omnetpp.ini

```

1 [General]
2 network = ARPTest
3 #abstract-config = true (requires omnet 7)
4 sim-time-limit = 500s
5 total-stack = 2MiB
6
7 [Config ARPTest]
8 # tcp apps
9 **.client.numApps = 1
10 **.client.app[*].typename = "TcpSessionApp"
11 **.client.app[*].active = true
12 **.client.app[*].localAddress = ""
13 **.client.app[*].localPort = -1
14 **.client.app[*].connectAddress = "server"
15 **.client.app[*].connectPort = 1000
16 **.client.app[*].tOpen = 1.0s

```

```

17 **.client.app[*].tSend = 1.1s
18 **.client.app[*].sendBytes = 1MiB
19 **.client.app[*].sendScript = ""
20 **.client.app[*].tClose = 0s
21
22 **.server.numApps = 1
23 **.server.app[*].typename = "TcpEchoApp"
24 **.server.app[0].localAddress = ""
25 **.server.app[0].localPort = 1000
26 **.server.app[0].echoFactor = 2.0
27 **.server.app[0].echoDelay = 0s
28
29 # Ethernet NIC configuration
30 **.eth[*].duplexMode = true
31
32 # Queues
33 **.ppp[*].queue.typename = "DropTailQueue"
34 **.ppp[*].queue.packetCapacity = 10
35 **.eth[*].queue.typename = "EthernetQosQueue"
36 **.eth[*].queue.dataQueue.typename = "DropTailQueue"
37 **.eth[*].queue.dataQueue.packetCapacity = 10
38
39 # Ethernet switch
40 **.switch*.bridging.typename = "MacRelayUnit"

```

Wildcard quick reference

- ****** matches any module path (recursive wildcard). Example: ****client** — any module named **client** anywhere.
- ***** inside an index like **app[*]** matches any array index (or all indices) of that vector.

line-by-line explanations

1) Global simulator settings

```

1 [General]
2 network = ARPTTest
3 #abstract-config = true (requires omnet 7)
4 sim-time-limit = 500s
5 total-stack = 2MiB

```

Explanation:

- **[General]** — section header for global simulator options.
- **network = ARPTTest** — choose the top-level NED network named **ARPTTest** to instantiate.
- **#abstract-config = true (requires omnet 7)** — commented-out reminder; if enabled on OMNeT++ 7+, switches config handling mode.
- **sim-time-limit = 500s** — stop simulation at 500 seconds of simulated time.
- **total-stack = 2MiB** — set total stack memory limit to 2 mebibytes (increase if you encounter stack overflows).

2) Begin named config (applies to ARPTTest)

```
1 [Config ARPTest]
```

Explanation:

- [Config ARPTest] — starts a configuration block named **ARPTest**; parameters below apply when this config is selected (matches the **network** above).

3) Create TCP client apps (count and type)

```
1 **.client.numApps = 1
2 **.client.app[*].typename = "TcpSessionApp"
3 **.client.app[*].active = true
```

Explanation:

- **.client.numApps = 1 — for any module named **client**, create an application array of size 1.
- **.client.app[*].typename = "TcpSessionApp" — instantiate **TcpSessionApp** for each app entry (the client application that initiates TCP sessions).
- **.client.app[*].active = true — make the app active (it will initiate outbound connections rather than passively waiting).

4) Client socket / addressing and ports

```
1 **.client.app[*].localAddress = ""
2 **.client.app[*].localPort = -1
3 **.client.app[*].connectAddress = "server"
4 **.client.app[*].connectPort = 1000
```

Explanation:

- localAddress = "" — bind to any local address (let the stack choose the interface/IP).
- localPort = -1 — use an ephemeral/local free port (do not force a specific source port).
- connectAddress = "server" — remote destination to connect to; resolves module named **server** (or an address mapping).
- connectPort = 1000 — remote TCP port on the server to connect to (must match server listener).

5) Client timing and send behavior

```
1 **.client.app[*].tOpen = 1.0s
2 **.client.app[*].tSend = 1.1s
3 **.client.app[*].sendBytes = 1MiB
4 **.client.app[*].sendScript = ""
5 **.client.app[*].tClose = 0s
```

Explanation:

- tOpen = 1.0s — schedule socket open (connection attempt) at 1.0 seconds.
- tSend = 1.1s — start sending data at 1.1 seconds (typically after connect completes).
- sendBytes = 1MiB — total payload to send: 1 mebibyte.
- sendScript = "" — no custom send script used; default/simple send behavior applies.

- `tClose = 0s` — close time semantics vary by app: often `0s` means “close immediately after finishing sends” or “no scheduled close”; check the app docs for exact behavior.

6) Create TCP server apps (count and type)

```
1 **.server.numApps = 1
2 **.server.app[*].typename = "TcpEchoApp"
```

Explanation:

- `**.`server.numApps = 1 — for modules named `server`, create an app array of size 1.
- `**.`server.app[*].typename = "TcpEchoApp" — instantiate `TcpEchoApp` (simple server that echoes received bytes).

7) Server binding and echo settings

```
1 **.server.app[0].localAddress = ""
2 **.server.app[0].localPort = 1000
3 **.server.app[0].echoFactor = 2.0
4 **.server.app[0].echoDelay = 0s
```

Explanation:

- `localAddress = ""` — server listens on all local interfaces (no single fixed IP).
- `localPort = 1000` — server listens on TCP port 1000 (client must connect to this).
- `echoFactor = 2.0` — application-specific: echo back data multiplied/replicated by factor 2.0 (consult INET docs for exact semantics).
- `echoDelay = 0s` — send the echo immediately (no extra server-side delay).

8) Ethernet NIC configuration

```
1 # Ethernet NIC configuration
2 **.eth[*].duplexMode = true
```

Explanation:

- `**.`eth[*].duplexMode = true — set all Ethernet interface instances named `eth[index]` to full duplex (allow simultaneous send/receive).

9) PPP queue settings (simple FIFO)

```
1 # Queues
2 **.ppp[*].queue.typename = "DropTailQueue"
3 **.ppp[*].queue.packetCapacity = 10
```

Explanation:

- `ppp[*].queue.typename = "DropTailQueue"` — use simple drop-tail FIFO queue for PPP interfaces.
- `packetCapacity = 10` — maximum 10 packets buffered; packets arriving when full are dropped at the tail.

10) Ethernet queue structure (QoS + data subqueue)

```

1 **.eth[*].queue.typename = "EthernetQosQueue"
2 **.eth[*].queue.dataQueue.typename = "DropTailQueue"
3 **.eth[*].queue.dataQueue.packetCapacity = 10

```

Explanation:

- `eth[*].queue.typename = "EthernetQosQueue"` — use a QoS-aware queue for Ethernet NICs (supports priorities/subqueues).
- `dataQueue.typename = "DropTailQueue"` — configure the data sub-queue as a drop-tail FIFO.
- `dataQueue.packetCapacity = 10` — limit the data sub-queue to 10 packets.

11) Ethernet switch bridging implementation

```

1 # Ethernet switch
2 **switch*.bridging.typename = "MacRelayUnit"

```

Explanation:

- `switch*.bridging.typename = "MacRelayUnit"` — for modules whose names start with `switch` (e.g., `switch0`), set their bridging submodule to `MacRelayUnit` (the forwarding/learning element implementing switch logic).

Part D: Modifying the IPv4 Addresses and adding static routing

If we use network configurator, we don't need to assign any ipv4 address or add any static routing, it will handle everything on its own. It uses dijkstra to calculate the static routes. But, for our own network we need to add custom ip addresses and routings. For that at first we need to add `addStaticRoutes = false;` in the configurator parameter. Then add the following in `.ini` file.

```

1 *.configurator.config = xml("<config> \
2   <interface hosts='client' names='eth*' address='192.168.1.10'
3     netmask='255.255.255.0' /> \
4   <interface hosts='host1' names='eth*' address='192.168.1.20'
5     netmask='255.255.255.0' /> \
6   <interface hosts='host2' names='eth*' address='192.168.1.30'
7     netmask='255.255.255.0' /> \
8   <interface hosts='router' names='eth*' address='192.168.1.1'
9     netmask='255.255.255.0' /> \
10  <interface hosts='router' names='ppp0' address='10.0.0.1' netmask
11    ='255.255.255.0' /> \
12  <interface hosts='net' names='ppp0' address='10.0.0.2' netmask
    ='255.255.255.0' /> \
    <interface hosts='net' names='ppp1' address='10.0.1.1'
      netmask='255.255.255.0' /> \
    <interface hosts='server' names='ppp*' address='10.0.1.2' netmask
      ='255.255.255.0' /> \
    <route hosts='client host1 host2' destination='*' gateway
      ='192.168.1.1' /> \
    <route hosts='server' destination='*' gateway='10.0.1.1' /> \

```

```

13     <route hosts='router' destination='10.0.1.0' netmask
        ='255.255.255.0' gateway='10.0.0.2'/> \
14     <route hosts='net' destination='192.168.1.0' netmask
        ='255.255.255.0' gateway='10.0.0.1'/> \
15 </config>")

```

Explanation

Network Topology Analysis Based on the IP addressing, the network is divided into three subnets.

Interface Configuration The `<interface>` tags define which IP addresses are assigned to which nodes and specific network cards (Ethernet `eth` or Point-to-Point `ppp`).

Node Name	Interface	IP Address	Subnet (Role)
client	eth*	192.168.1.10	LAN (End User)
host1	eth*	192.168.1.20	LAN (End User)
host2	eth*	192.168.1.30	LAN (End User)
router	eth*	192.168.1.1	LAN (Gateway)
router	ppp0	10.0.0.1	WAN Link 1
net	ppp0	10.0.0.2	WAN Link 1
net	ppp1	10.0.1.1	WAN Link 2
server	ppp*	10.0.1.2	Server Network

Table 1: IP Address Assignments

Routing Logic The `<route>` tags define how traffic moves between these networks.

- **LAN Hosts (client, host1, host2):**
 1. `destination='*'`: Uses a default route.
 2. `gateway='192.168.1.1'`: All traffic leaving the LAN is sent to the **router**.
- **Server:**
 1. `destination='*'`: Uses a default route.
 2. `gateway='10.0.1.1'`: All return traffic is sent to the **net** node.
- **Router:**
 1. Knows the LAN (192.168.1.0) is directly connected.
 2. Needs a static route to reach the Server network (10.0.1.0).
 3. Forwards traffic for 10.0.1.0 to gateway 10.0.0.2 (the 'net' node).
- **Net Node (Intermediate Router):**
 1. Knows the Server network (10.0.1.0) is directly connected.
 2. Needs a static route to reach the Client LAN (192.168.1.0).
 3. Forwards traffic for 192.168.1.0 to gateway 10.0.0.1 (the 'router').